Lógica: inducción y recurrencias

Bloque 2: sumatorias, funciones recursivas y su costo

Carlos Andrés Delgado Saavedra

Pontificia Universidad Javeriana Cali

Objetivos

Al terminar, usted podrá

- Demostrar por inducción las fórmulas cerradas de $\sum 1$, $\sum i$ y $\sum i^2$ que se usan al contar operaciones.
- Demostrar que una función recursiva por índices devuelve lo que promete, con inducción sobre el tamaño del rango.
- Plantear la recurrencia $T(n)$ de un programa recursivo.
- Resolver $T(n) = T(n-1) + c$, $T(n) = T(n/2) + c$ y $T(n) = 2T(n/2) + cn$ por sustitución repetida y con el árbol de recursión, y comprobar la forma cerrada por inducción.

Conocimiento previo

Del conteo de operaciones

Un ciclo que depende del índice externo se cuenta con una sumatoria, como $\sum_{i=0}^{n-1} i$ en un ciclo interno que corre mientras $j < i$.

De la recursión por índices

Una función que recibe el rango $[ini, fin)$ de un arreglo y se llama a sí misma sobre un rango más pequeño, sin copiar.

De la definición de O

$f(n) \in O(g(n))$ si existen $c > 0$ y $k \geq 0$ tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq k$. Toda demostración va en cuatro partes: teorema, estrategia, desarrollo y conclusión.

Contenido

1 Inducción sobre las sumatorias del conteo

2 Correctitud de una función recursiva

3 Recurrencias: plantear y resolver

4 Referencias

Contenido

1 Inducción sobre las sumatorias del conteo
- Concepto
- Ejemplo
- Ejercicio

2 Correctitud de una función recursiva

3 Recurrencias: plantear y resolver

4 Referencias

Una fórmula para infinitos valores de n

Pregunta

¿Vale $1+2+\cdots+n=\dfrac{n(n+1)}{2}$ para todo $n \geq 1$?

n	$1+2+\cdots+n$	$n(n+1)/2$	¿Coinciden?
1	1	1	sí
2	3	3	sí
3	6	6	sí
4	10	10	sí
5	15	15	sí

Cinco filas no son todos los n

La tabla revisa cinco valores. Para $n=6$, $n=1000$ y los demás hace falta un argumento que no dependa de un n particular.

El principio de inducción matemática

Definición

Para demostrar que una propiedad $P(n)$ vale para todo entero $n \geq n_0$ basta con dos cosas:

- **Base**: $P(n_0)$ es verdadera.
- **Paso**: para todo $k \geq n_0$, $P(k) \to P(k+1)$.

En el paso se supone $P(k)$, que se llama *hipótesis de inducción* (HI), y con ella se deduce $P(k+1)$.

Cómo se encadena

La base da $P(1)$. El paso con $k = 1$ da $P(2)$; con $k = 2$, $P(3)$; y así para cada entero. Ningún $n \geq 1$ queda por fuera.

Una demostración por inducción, en cuatro partes

Las cuatro partes

Teorema: la fórmula y el rango de n. Estrategia: *se procede por inducción sobre* n. Desarrollo: base, hipótesis y paso. Conclusión: la fórmula vale para todo n del rango.

El primer caso: $\sum_{i=1}^{n} 1 = n$

Teorema

Para todo $n \geq 1$, $\displaystyle\sum_{i=1}^{n} 1 = n$.

Estrategia

Se procede por inducción sobre n.

Desarrollo

Base ($n = 1$): la suma tiene un solo término, $\sum_{i=1}^{1} 1 = 1$.

El primer caso: $\sum_{i=1}^{n} 1 = n$

Desarrollo, continuación

Hipótesis: para un $k \geq 1$, $\sum_{i=1}^{k} 1 = k$.
Paso: se separa el último término y se usa la hipótesis:
$\sum_{i=1}^{k+1} 1 = \sum_{i=1}^{k} 1 + 1 \overset{\text{HI}}{=} k + 1$.

Conclusión

Por inducción, $\sum_{i=1}^{n} 1 = n$ para todo $n \geq 1$. Es la cuenta de las vueltas de un ciclo de n iteraciones.

$\sum_{i=1}^{n} i = n(n+1)/2$

Teorema

Para todo $n \geq 1$, $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Estrategia

Se procede por inducción sobre n.

Pregunta

¿Qué hay que comprobar en la base?

$\sum_{i=1}^{n} i = n(n+1)/2$

Teorema

Para todo $n \geq 1$, $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Estrategia

Se procede por inducción sobre n.

Base ($n = 1$)

A la izquierda, $\sum_{i=1}^{1} i = 1$. A la derecha, $\frac{1 \cdot 2}{2} = 1$. Los dos lados valen 1.

Hipótesis

Para un $k \geq 1$ fijo, $\displaystyle\sum_{i=1}^{k} i = \frac{k(k+1)}{2}$.

$\sum_{i=1}^{n} i = n(n+1)/2$: el paso

Hay que llegar a

$$\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}.$$

Pregunta

La hipótesis habla de la suma hasta k. ¿Qué término se separa de la suma hasta $k+1$ para que la hipótesis aparezca?

$\sum_{i=1}^{n} i = n(n+1)/2$: el paso

Paso: hay que llegar a $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$

Se separa el último término, $i = k+1$:

$$\sum_{i=1}^{k+1} i = \sum_{i=1}^{k} i + (k+1) \qquad \text{término } k+1$$

$$\overset{\text{HI}}{=} \frac{k(k+1)}{2} + (k+1) \qquad \text{por HI}$$

$$= \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2} \qquad \text{factor } (k+1)$$

Conclusión

La base vale y el paso vale para todo $k \geq 1$. Por inducción, $\sum_{i=1}^{n} i = n(n+1)/2$ para todo $n \geq 1$.

La forma que aparece al contar: de 0 a $n-1$

Pregunta

Un ciclo interno corre mientras $j < i$, con i de 0 a $n-1$. ¿Cuántas veces corre su cuerpo con $n = 4$ y con n cualquiera?

La forma que aparece al contar: de 0 a $n-1$

Pregunta

Un ciclo interno corre mientras $j < i$, con i de 0 a $n-1$. ¿Cuántas veces corre su cuerpo con $n = 4$ y con n cualquiera?

i	0	1	2	3	total
vueltas del cuerpo	0	1	2	3	6

Con el teorema anterior

Para $n \geq 1$, el término $i = 0$ suma cero y el resto es la fórmula con $n-1$ en lugar de n:

$$\sum_{i=0}^{n-1} i \;=\; 0 + \sum_{i=1}^{n-1} i \;=\; \frac{(n-1)n}{2}.$$

Con $n = 4$: $\frac{3 \cdot 4}{2} = 6$, lo mismo que la tabla. Con $n = 1$ la suma de 1 a 0 no tiene términos y vale $0 = \frac{0 \cdot 1}{2}$.

Un paso que funciona no basta

Pregunta

Alguien propone $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} + 1$ y escribe el paso:

$$\sum_{i=1}^{k+1} i \overset{\text{HI}}{=} \frac{k(k+1)}{2} + 1 + (k+1) = \frac{(k+1)(k+2)}{2} + 1.$$

La cuenta está bien. ¿La fórmula es verdadera?

Un paso que funciona no basta

Pregunta

Alguien propone $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} + 1$ y escribe el paso:

$$\sum_{i=1}^{k+1} i \overset{\text{HI}}{=} \frac{k(k+1)}{2} + 1 + (k+1) = \frac{(k+1)(k+2)}{2} + 1.$$

La cuenta está bien. ¿La fórmula es verdadera?

No: la base falla

Con $n = 1$ la izquierda vale 1 y la derecha $\frac{1\cdot 2}{2} + 1 = 2$. El paso dice que *si* la fórmula valiera para k, valdría para $k+1$; como no vale para ningún k desde el cual arrancar, la cadena nunca empieza. La base no se omite.

Errores comunes en una inducción

Suponer lo que se quiere demostrar

La hipótesis es $P(k)$ para un k fijo. Escribir "supongamos que vale para todo n" es suponer el teorema.

Llegar al resultado desde ambos lados

El paso parte de un lado de $P(k+1)$ y llega al otro con igualdades justificadas. Partir de $P(k+1)$ como si fuera cierta y llegar a $0 = 0$ no demuestra nada.

Un paso sin la hipótesis

Si el paso nunca usa HI, o la cuenta tiene un error, o lo que se hizo no es una inducción. La marca $\overset{\text{HI}}{=}$ muestra dónde se usa.

Ejercicio 1 (intermedio): la suma de cuadrados

Enunciado

1. Demuestre en cuatro partes que para todo $n \geq 1$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}.$$

2. Deduzca de la anterior, para $n \geq 1$, la forma del conteo:
$$\sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6}.$$

3. En *UVa 12149, Feynman* (`https://onlinejudge.org/external/121/12149.pdf`) una cuadrícula de $N \times N$ tiene $\sum_{m=1}^{N} m^2$ cuadrados. ¿Cuántos hay con $N = 8$?

Ejercicio 1: dónde se atasca el paso

Pregunta

Tras separar el último término y usar la hipótesis queda

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2.$$

¿Qué factor comparten los dos sumandos, y qué polinomio de grado 2 queda al sacarlo?

Ejercicio 1: dónde se atasca el paso

Pregunta

Tras separar el último término y usar la hipótesis queda

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2.$$

¿Qué factor comparten los dos sumandos, y qué polinomio de grado 2 queda al sacarlo?

Respuesta

El factor es $(k+1)$, y queda $\frac{(k+1)\,[\,k(2k+1)+6(k+1)\,]}{6} = \frac{(k+1)(2k^2+7k+6)}{6}$. El objetivo pide $(k+2)(2k+3)$: hay que factorizar $2k^2+7k+6$.

Solución del ejercicio 1, punto 1

Teorema

Para todo $n \geq 1$, $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$.

Estrategia

Se procede por inducción sobre n.

Base ($n = 1$)

$\sum_{i=1}^{1} i^2 = 1$ y $\frac{1 \cdot 2 \cdot 3}{6} = 1$.

Hipótesis

Para un $k \geq 1$ fijo, $\sum_{i=1}^{k} i^2 = \frac{k(k+1)(2k+1)}{6}$.

Solución del ejercicio 1, punto 1

Paso

$$\sum_{i=1}^{k+1} i^2 = \sum_{i=1}^{k} i^2 + (k+1)^2 \overset{\text{HI}}{=} \frac{k(k+1)(2k+1)}{6} + (k+1)^2$$
$$= \frac{(k+1)\left[2k^2+k+6k+6\right]}{6} = \frac{(k+1)(2k^2+7k+6)}{6}$$
$$= \frac{(k+1)(k+2)(2k+3)}{6}$$

La última igualdad usa $(k+2)(2k+3) = 2k^2+7k+6$. Y $(k+1)(k+2)(2k+3)$ es $n(n+1)(2n+1)$ con $n = k+1$.

Conclusión

Por inducción, $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ para todo $n \geq 1$.

Solución del ejercicio 1, puntos 2 y 3

Punto 2

El término $i = 0$ suma cero, y el resto es el teorema con $n - 1$:

$$\sum_{i=0}^{n-1} i^2 = \sum_{i=1}^{n-1} i^2 = \frac{(n-1)\,n\,(2(n-1)+1)}{6} = \frac{(n-1)n(2n-1)}{6}.$$

Con $n = 1$ la suma no tiene términos y la fórmula da 0.

Punto 3

Con $N = 8$: $\frac{8 \cdot 9 \cdot 17}{6} = 204$ cuadrados. Es la salida que trae el enunciado para esa entrada, y la fórmula la da sin recorrer la cuadrícula.

Contenido

1 Inducción sobre las sumatorias del conteo

2 Correctitud de una función recursiva
- Concepto
- Ejemplo
- Ejercicio

3 Recurrencias: plantear y resolver

4 Referencias

Qué se demuestra de una función recursiva

Especificación

Una fórmula que dice qué devuelve la función en términos de sus argumentos. Para la suma de un rango: $suma(A, ini, fin) = \sum_{j=ini}^{fin-1} A[j]$.

Inducción sobre el tamaño de la entrada

Se escoge una medida que baje en cada llamada; con rangos, $m = fin - ini$.

- **Base**: el caso base de la función. Ahí no hay llamadas y se revisa el valor que devuelve.
- **Hipótesis**: la función cumple la especificación para *toda* entrada de tamaño menor que m.
- **Paso**: el caso recursivo con tamaño m. Cada llamada recibe una entrada más pequeña, así que la hipótesis dice qué devuelve; con eso se comprueba que el resultado cumple la especificación.

Inducción fuerte

Definición

Para demostrar $P(m)$ para todo $m \geq m_0$:

- **Base**: $P(m_0)$.
- **Paso**: para todo $m > m_0$, $\big(\forall j,\ m_0 \leq j < m :\ P(j)\big) \to P(m)$.

La hipótesis cubre todos los tamaños menores que m, no solo $m-1$.

Ejemplo: cuándo hace falta

Si la función se llama sobre la mitad del rango, con $m = 10$ la llamada recibe tamaño 5, no 9. La hipótesis simple, sobre $m-1$, no dice nada de esa llamada; la fuerte sí, porque $5 < 10$.

La suma de un rango, por índices

```python
def suma(A, ini, fin):
    res = 0
    if ini < fin:
        res = A[ini] + suma(A, ini + 1, fin)
    return res
```

```java
int suma(int A[], int ini, int fin) {
  int res = 0;
  if (ini < fin) {
    res = A[ini] + suma(A, ini + 1, fin);
  }
  return res;
}
```

Pregunta

Con $A = [4, 1, 3]$, ¿qué llamadas hace $suma(A, 0, 3)$ y qué devuelve cada una?

La suma de un rango: traza

Pregunta

Con $A = [4, 1, 3]$, ¿qué llamadas hace $suma(A, 0, 3)$ y qué devuelve cada una?

Llamada	*ini*	*fin*	$m = fin - ini$	Devuelve
$suma(A, 0, 3)$	0	3	3	$4 + 4 = 8$
$suma(A, 1, 3)$	1	3	2	$1 + 3 = 4$
$suma(A, 2, 3)$	2	3	1	$3 + 0 = 3$
$suma(A, 3, 3)$	3	3	0	0, caso base

Las respuestas suben de la última fila a la primera: cada llamada usa lo que devolvió la de abajo.

¿Sobre qué se hace la inducción?

Pregunta

Hay tres candidatos: n, la longitud del arreglo; ini; y $m = fin - ini$. ¿Cuál baja en cada llamada y llega al caso base?

¿Sobre qué se hace la inducción?

Pregunta

Hay tres candidatos: n, la longitud del arreglo; ini; y $m = fin - ini$. ¿Cuál baja en cada llamada y llega al caso base?

Respuesta: m

- n no cambia: todas las llamadas reciben el mismo A.
- ini sube, y el caso base no depende solo de ini, sino de que ini alcance a fin.
- m baja de uno en uno, de 3 a 0 en la traza, y $m = 0$ es exactamente el caso base de la función.

Correctitud de `suma`

Teorema

Para todo $m \geq 0$ y todo ini con $0 \leq ini$ e $ini + m \leq n$:

$$suma(A, ini, ini + m) = \sum_{j=ini}^{ini+m-1} A[j].$$

Estrategia

Se procede por inducción sobre $m = fin - ini$.

Base ($m = 0$)

Entonces $ini = fin$, la condición $ini < fin$ es falsa y la función devuelve $res = 0$. La suma $\sum_{j=ini}^{ini-1} A[j]$ no tiene términos y vale 0.

Correctitud de `suma`

Hipótesis

Para un $k \geq 0$ fijo y todo *ini* válido, $\mathit{suma}(A, \mathit{ini}, \mathit{ini} + k) = \sum_{j=\mathit{ini}}^{\mathit{ini}+k-1} A[j]$.

Paso ($m = k + 1$)

Sea $\mathit{fin} = \mathit{ini} + k + 1$. Como $\mathit{ini} < \mathit{fin}$, la función toma la rama recursiva. La llamada $\mathit{suma}(A, \mathit{ini} + 1, \mathit{fin})$ tiene tamaño $\mathit{fin} - (\mathit{ini} + 1) = k$, así que

$$\mathit{res} = A[\mathit{ini}] + \mathit{suma}(A, \mathit{ini} + 1, \mathit{fin}) \overset{\text{HI}}{=} A[\mathit{ini}] + \sum_{j=\mathit{ini}+1}^{\mathit{fin}-1} A[j] = \sum_{j=\mathit{ini}}^{\mathit{fin}-1} A[j].$$

Conclusión

Por inducción sobre m, $\mathit{suma}(A, \mathit{ini}, \mathit{fin})$ devuelve $\sum_{j=\mathit{ini}}^{\mathit{fin}-1} A[j]$ para todo rango válido. En particular $\mathit{suma}(A, 0, n)$ es la suma del arreglo.

Errores comunes al demostrar una función recursiva

Inducir sobre lo que no cambia

Con inducción sobre n, la hipótesis habla de arreglos más cortos, y la llamada recibe el mismo arreglo. La hipótesis no se puede aplicar.

Aplicar la hipótesis al mismo tamaño

En el paso, la hipótesis solo cubre entradas *más pequeñas*. Usarla sobre una llamada de tamaño m es suponer lo que se quiere demostrar.

Una base que no es la de la función

La base de la inducción es el caso base del código. Si la función para en $fin - ini = 1$, la inducción arranca en $m = 1$, y el teorema no dice nada del rango vacío.

Ejercicio 2: dos funciones por demostrar

Enunciado

1. (Intermedio) $maximo(A, ini, fin)$ recibe un rango no vacío, $fin - ini \geq 1$. Escriba su especificación como fórmula y demuestre por inducción que la cumple.
2. (Avanzado) $sumaMitades(A, ini, fin)$ parte el rango por la mitad. Demuestre que devuelve $\sum_{j=ini}^{fin-1} A[j]$. ¿Qué tamaños reciben sus llamadas, y qué tipo de inducción pide eso?

El código de las dos va en las páginas siguientes.

Ejercicio 2: `maximo` en Python

```
def maximo(A, ini, fin):
    if fin - ini == 1:
        res = A[ini]
    else:
        otro = maximo(A, ini + 1, fin)
        if A[ini] > otro:
            res = A[ini]
        else:
            res = otro
    return res
```

Con $A = [2, 7, 5]$, $maximo(A, 0, 3)$ devuelve 7.

Ejercicio 2: maximo en C++

```
int maximo(int A[], int ini, int fin) {
  int res, otro;
  if (fin - ini == 1) {
    res = A[ini];
  } else {
    otro = maximo(A, ini + 1, fin);
    if (A[ini] > otro) {
      res = A[ini];
    } else {
      res = otro;
    }
  }
  return res;
}
```

Con $A = [2, 7, 5]$, $maximo(A, 0, 3)$ devuelve 7.

Ejercicio 2: sumaMitades en Python

```
def sumaMitades(A, ini, fin):
    if fin - ini == 0:
        res = 0
    elif fin - ini == 1:
        res = A[ini]
    else:
        mid = (ini + fin) // 2
        res = (sumaMitades(A, ini, mid)
               + sumaMitades(A, mid, fin))
    return res
```

Con $A = [4, 1, 3, 2]$, $sumaMitades(A, 0, 4)$ devuelve 10.

Ejercicio 2: sumaMitades en C++

```
int sumaMitades(int A[], int ini, int fin) {
  int res, mid;
  if (fin - ini == 0) {
    res = 0;
  } else if (fin - ini == 1) {
    res = A[ini];
  } else {
    mid = (ini + fin) / 2;
    res = sumaMitades(A, ini, mid)
        + sumaMitades(A, mid, fin);
  }
  return res;
}
```

Con $A = [4, 1, 3, 2]$, $sumaMitades(A, 0, 4)$ devuelve 10.

Ejercicio 2: la pregunta del punto 2

Pregunta

Con $m = fin - ini \geq 2$ y $mid = \lfloor (ini + fin)/2 \rfloor$, ¿qué tamaños tienen $[ini, mid)$ y $[mid, fin)$? ¿Son menores que m?

Ejercicio 2: la pregunta del punto 2

Pregunta

Con $m = fin - ini \geq 2$ y $mid = \lfloor (ini + fin)/2 \rfloor$, ¿qué tamaños tienen $[ini, mid)$ y $[mid, fin)$? ¿Son menores que m?

Respuesta

$mid - ini = \lfloor m/2 \rfloor$ y $fin - mid = m - \lfloor m/2 \rfloor = \lceil m/2 \rceil$. Con $m \geq 2$ los dos están entre 1 y $m - 1$. Ninguno es $m - 1$ en general: con $m = 10$ son 5 y 5. Hace falta la inducción fuerte.

m	$\lfloor m/2 \rfloor$	$\lceil m/2 \rceil$	¿Ambos en $[1, m)$?
2	1	1	sí
3	1	2	sí
10	5	5	sí

Solución del ejercicio 2, punto 1

Teorema

Para todo $m \geq 1$ y todo rango $[ini, fin)$ válido con $fin - ini = m$, el valor $r = maximo(A, ini, fin)$ cumple

$$\big(\exists j,\ ini \leq j < fin :\ r = A[j]\big)\ \wedge\ \big(\forall j,\ ini \leq j < fin :\ A[j] \leq r\big).$$

Estrategia

Se procede por inducción sobre $m = fin - ini$, desde $m = 1$, que es el caso base de la función.

Base ($m = 1$)

La función devuelve $r = A[ini]$. El testigo del existencial es $j = ini$, y el único j del rango es ini, con $A[ini] \leq r$.

Solución del ejercicio 2, punto 1

Hipótesis

Para un $k \geq 1$ fijo, la llamada sobre todo rango de tamaño k cumple la especificación.

Paso ($m = k + 1 \geq 2$)

La función toma la rama recursiva. $[ini + 1, fin)$ tiene tamaño k; por HI, *otro* es un $A[j']$ con $ini < j' < fin$ y $A[j] \leq otro$ para todo j con $ini < j < fin$.

- Si $A[ini] > otro$: $r = A[ini]$, testigo $j = ini$. Para $j > ini$, $A[j] \leq otro < r$; y $A[ini] \leq r$.
- Si no: $r = otro$, testigo j'. Para $j > ini$, $A[j] \leq r$ por HI; y $A[ini] \leq otro = r$ porque la condición fue falsa.

Conclusión

Por inducción, $maximo(A, ini, fin)$ devuelve el máximo de todo rango no vacío.

Solución del ejercicio 2, punto 2

Teorema y estrategia

Para todo $m \geq 0$ y todo rango válido con $fin - ini = m$, $sumaMitades(A, ini, fin) = \sum_{j=ini}^{fin-1} A[j]$. Se procede por inducción fuerte sobre m.

Base ($m = 0$ y $m = 1$)

Los dos casos base de la función: con $m = 0$ devuelve 0, la suma vacía; con $m = 1$ devuelve $A[ini]$, la suma de un término.

Solución del ejercicio 2, punto 2

Hipótesis y paso ($m \geq 2$)

HI: la función cumple para todo tamaño j con $0 \leq j < m$. Los rangos $[ini, mid)$ y $[mid, fin)$ tienen tamaños $\lfloor m/2 \rfloor$ y $\lceil m/2 \rceil$, ambos menores que m, así que

$$res \overset{\text{HI}}{=} \sum_{j=ini}^{mid-1} A[j] + \sum_{j=mid}^{fin-1} A[j] = \sum_{j=ini}^{fin-1} A[j].$$

Conclusión

Por inducción fuerte, *sumaMitades* devuelve la suma de todo rango válido.

Contenido

1 Inducción sobre las sumatorias del conteo

2 Correctitud de una función recursiva

3 Recurrencias: plantear y resolver
- Concepto
- Ejemplo: $T(n) = T(n-1) + c$
- Ejemplo: $T(n) = T(n/2) + c$
- Ejemplo: $T(n) = 2T(n/2) + cn$
- Ejercicio

4 Referencias

Plantear $T(n)$ de un programa recursivo

Una ecuación por caso del código

- Caso base: no hay llamadas y el trabajo es constante, d.
- Caso recursivo: la suma de T sobre el tamaño de cada llamada, más el trabajo que la función hace fuera de las llamadas.

Ejemplo: `suma`

Con $m = fin - ini$: el caso base cuesta d; el recursivo hace una llamada de tamaño $m - 1$ y, fuera de ella, una comparación, una suma y una asignación, que cuestan una constante c.

$$T(0) = d, \qquad T(m) = T(m-1) + c \quad \text{para } m \geq 1.$$

Tres recurrencias, tres programas

Programa	Recurrencia
`suma` por índices; un móvil de *UVa 839* en el que cada nivel cuelga un solo submóvil	$T(n) = T(n-1) + c$
El código Gray de la posición k en *UVa 11173*: una llamada sobre la mitad del rango de posiciones	$T(n) = T(n/2) + c$
Ordenar por mezcla: dos llamadas sobre las mitades y una mezcla que recorre el rango	$T(n) = 2T(n/2) + cn$

- *UVa 839, Not so Mobile*: `https://onlinejudge.org/external/8/839.pdf`
- *UVa 11173, Grey Codes*: `https://onlinejudge.org/external/111/11173.pdf`

Dos formas de resolver una recurrencia

Sustitución repetida

Se reemplaza el término $T(\cdot)$ de la derecha por su propia definición, una y otra vez, hasta ver el patrón después de i sustituciones; se escoge el i que llega al caso base.

Árbol de recursión

Cada llamada es un nodo con el trabajo que hace fuera de sus llamadas. Se suma cada nivel, se cuentan los niveles y se suman las hojas.

Comprobación por inducción

Los dos métodos producen una forma cerrada a partir de un patrón observado. La forma cerrada se demuestra por inducción, con la recurrencia en el paso.

$T(n) = T(n-1) + c$, con $T(0) = d$

Sustitución repetida

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= \big(T(n-2) + c\big) + c = T(n-2) + 2c \\ &= \big(T(n-3) + c\big) + 2c = T(n-3) + 3c \end{aligned}$$

Pregunta

¿Qué queda después de i sustituciones?

$T(n) = T(n-1) + c$, con $T(0) = d$

Sustitución repetida

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= \big(T(n-2) + c\big) + c = T(n-2) + 2c \\ &= \big(T(n-3) + c\big) + 2c = T(n-3) + 3c \\ &\vdots \\ &= T(n-i) + ic \end{aligned}$$

Pregunta

¿Cuándo se detienen las sustituciones?

$T(n) = T(n-1) + c$, con $T(0) = d$

Sustitución repetida

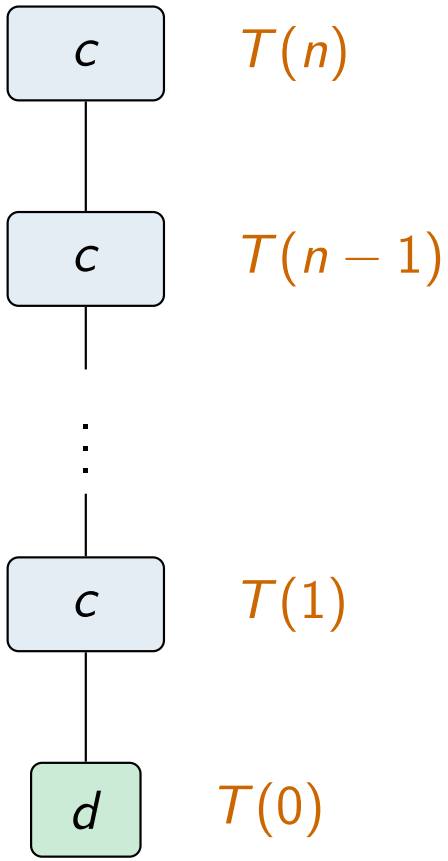
$$\begin{aligned} T(n) &= T(n-1) + c \\ &= \big(T(n-2) + c\big) + c = T(n-2) + 2c \\ &= \big(T(n-3) + c\big) + 2c = T(n-3) + 3c \\ &\vdots \\ &= T(n-i) + ic \end{aligned}$$

Respuesta

Cuando se llega a $T(0)$, es decir, con $n - i = 0$, o sea $i = n$:

$$T(n) = T(0) + nc = cn + d.$$

$T(n) = T(n-1) + c$: el árbol es una cadena



Lectura

Cada llamada hace una sola llamada, así que cada nivel tiene un nodo. Hay n nodos de costo c, de $T(n)$ a $T(1)$, y una hoja de costo d.

$$T(n) = \underbrace{c + c + \cdots + c}_{n \text{ veces}} + d = cn + d.$$

Comprobación con $c = 3$, $d = 2$

$T(0) = 2$, $T(1) = 5$, $T(2) = 8$, $T(3) = 11$, y $3 \cdot 3 + 2 = 11$.

$T(n) = cn + d$: comprobación por inducción

Teorema

Si $T(0) = d$ y $T(n) = T(n-1) + c$ para $n \geq 1$, entonces $T(n) = cn + d$ para todo $n \geq 0$.

Estrategia

Se procede por inducción sobre n.

Desarrollo

Base ($n = 0$): $T(0) = d = c \cdot 0 + d$.
Hipótesis: para un $k \geq 0$ fijo, $T(k) = ck + d$.

Paso: $T(k+1) = T(k) + c \overset{\text{HI}}{=} ck + d + c = c(k+1) + d$, donde la primera igualdad es la recurrencia con $n = k + 1 \geq 1$.

Conclusión

Por inducción, $T(n) = cn + d$ para todo $n \geq 0$.

De la forma cerrada a O

Teorema

$cn + d \in O(n)$, con $c > 0$ y $d \geq 0$ constantes.

Estrategia

Demostración directa: acotar d por un múltiplo de n.

Desarrollo

Se toman como testigos $c + d$ y $k = 1$. Si $n \geq 1$, multiplicar $1 \leq n$ por $d \geq 0$ da $d \leq dn$, y entonces $cn + d \leq cn + dn = (c + d)\, n$.

Conclusión

Con los testigos $c + d$ y $k = 1$, $cn + d \in O(n)$. La función `suma` hace un trabajo lineal en el tamaño del rango, y la de *UVa 839* también cuando cada nivel del móvil cuelga un solo submóvil.

$T(n) = T(n/2) + c$, con $T(1) = d$

Sustitución repetida, con $n = 2^j$ para que las mitades sean enteras

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + 2c \\ &= T(n/8) + 3c \\ &\vdots \\ &= T(n/2^i) + ic \end{aligned}$$

Pregunta

¿Cuándo se detienen las sustituciones, y cuántas hubo?

$T(n) = T(n/2) + c$, con $T(1) = d$

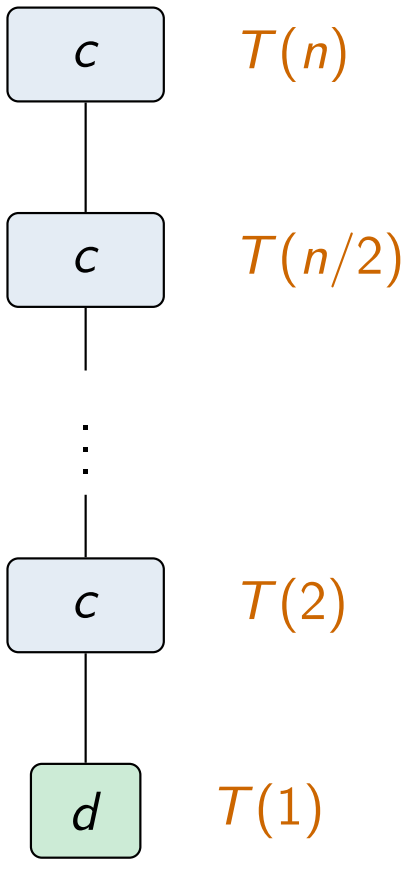
Sustitución repetida, con $n = 2^j$ para que las mitades sean enteras

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + 2c \\ &= T(n/8) + 3c \\ &\vdots \\ &= T(n/2^i) + ic \end{aligned}$$

Respuesta

Al llegar a $T(1)$: $n/2^i = 1$, o sea $2^i = n$ e $i = \lg n$. Entonces $T(n) = T(1) + c \lg n = c \lg n + d$.

$T(n) = T(n/2) + c$: una cadena de $\lg n + 1$ nodos



Lectura

El árbol tiene la misma forma que el de $T(n-1)+c$, pero ahora el tamaño se divide en vez de restar uno. De n a 1 hay $\lg n$ divisiones: $\lg n$ nodos de costo c y una hoja d.

Con $c = d = 1$:

n	1	2	4	8	1024
$\lg n$	0	1	2	3	10
$T(n)$	1	2	3	4	11

$T(n) = c \lg n + d$: comprobación por inducción

Teorema

Si $T(1) = d$ y $T(n) = T(n/2) + c$ para $n \geq 2$ potencia de 2, entonces $T(2^j) = cj + d$ para todo $j \geq 0$; es decir, $T(n) = c \lg n + d$.

Estrategia

Se procede por inducción sobre el exponente j, porque la recurrencia pasa de 2^{j+1} a 2^j.

Desarrollo

Base ($j = 0$): $T(2^0) = T(1) = d = c \cdot 0 + d$.
Hipótesis: para un $j \geq 0$ fijo, $T(2^j) = cj + d$.
Paso: $T(2^{j+1}) = T(2^{j+1}/2) + c = T(2^j) + c \overset{\text{HI}}{=} cj + d + c = c(j+1) + d$.

Conclusión

Por inducción, $T(2^j) = cj + d$ para todo $j \geq 0$, y con $j = \lg n$, $T(n) = c \lg n + d$.

El código Gray de *UVa 11173*

Pregunta

Con n bits hay $m = 2^n$ posiciones. La función baja a la mitad del rango de posiciones en cada llamada. ¿Cuánto cuesta, en términos de n?

El código Gray de *UVa 11173*

Pregunta

Con n bits hay $m = 2^n$ posiciones. La función baja a la mitad del rango de posiciones en cada llamada. ¿Cuánto cuesta, en términos de n?

Respuesta

La recurrencia es $T(m) = T(m/2) + c$ con $m = 2^n$, una potencia de 2. Por el teorema anterior,

$$T(2^n) = cn + d.$$

El costo crece con el número de bits, no con las 2^n posiciones. Construir la lista entera de códigos costaría al menos 2^n pasos.

Plantear la recurrencia de la mezcla

```
def ordenar(A, ini, fin):
    if fin - ini > 1:
        mid = (ini + fin) // 2
        ordenar(A, ini, mid)
        ordenar(A, mid, fin)
        mezclar(A, ini, mid, fin)    # recorre [ini, fin) una vez
```

Pregunta

Con $n = fin - ini$ potencia de 2, ¿cuántas llamadas hace, de qué tamaño, y cuánto trabajo queda fuera de ellas?

Plantear la recurrencia de la mezcla

```python
def ordenar(A, ini, fin):
    if fin - ini > 1:
        mid = (ini + fin) // 2
        ordenar(A, ini, mid)
        ordenar(A, mid, fin)
        mezclar(A, ini, mid, fin)    # recorre [ini, fin) una vez
```

Pregunta

Con $n = fin - ini$ potencia de 2, ¿cuántas llamadas hace, de qué tamaño, y cuánto trabajo queda fuera de ellas?

Respuesta

Dos llamadas de tamaño $n/2$; fuera de ellas, la mezcla recorre los n elementos con trabajo constante por elemento:

$$T(1) = d, \qquad T(n) = 2T(n/2) + cn \quad \text{para } n \geq 2.$$

$T(n) = 2T(n/2) + cn$: sustitución repetida

Sustitución repetida

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2\big(2T(n/4) + cn/2\big) + cn = 4T(n/4) + 2cn \\ &= 4\big(2T(n/8) + cn/4\big) + 2cn = 8T(n/8) + 3cn \end{aligned}$$

Pregunta

¿Qué queda después de i sustituciones? Mire el coeficiente de T y el de cn.

$T(n) = 2T(n/2) + cn$: sustitución repetida

Sustitución repetida

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2\big(2T(n/4) + cn/2\big) + cn = 4T(n/4) + 2cn \\ &= 4\big(2T(n/8) + cn/4\big) + 2cn = 8T(n/8) + 3cn \\ &\vdots \\ &= 2^i\ T(n/2^i) + i\,cn \end{aligned}$$

Hasta el caso base

Con $i = \lg n$ se llega a $T(1)$, y $2^{\lg n} = n$:

$$T(n) = n\,T(1) + cn \lg n = cn \lg n + dn.$$

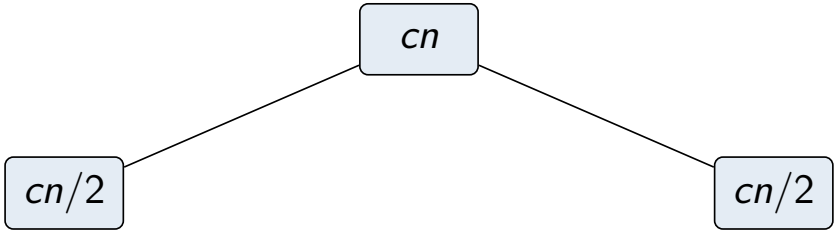
$T(n) = 2T(n/2) + cn$: el árbol, nivel 0



Pregunta

La raíz trabaja cn fuera de sus dos llamadas. ¿Cuánto trabaja cada hijo?

$T(n) = 2T(n/2) + cn$: el árbol, nivel 1



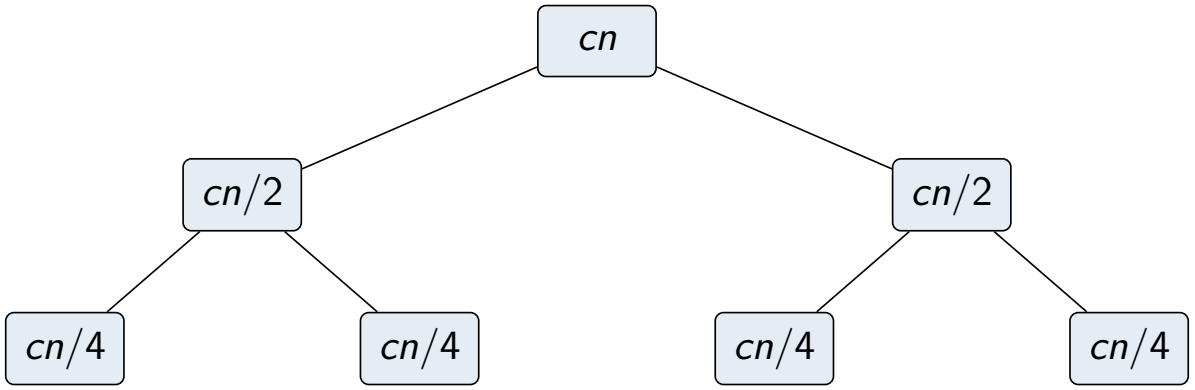
cn

$2 \cdot cn/2 = cn$

Pregunta

Cada hijo recibe $n/2$ elementos y trabaja $cn/2$. El nivel suma cn otra vez. ¿Y el nivel 2?

$T(n) = 2T(n/2) + cn$: el árbol, nivel 2



cn

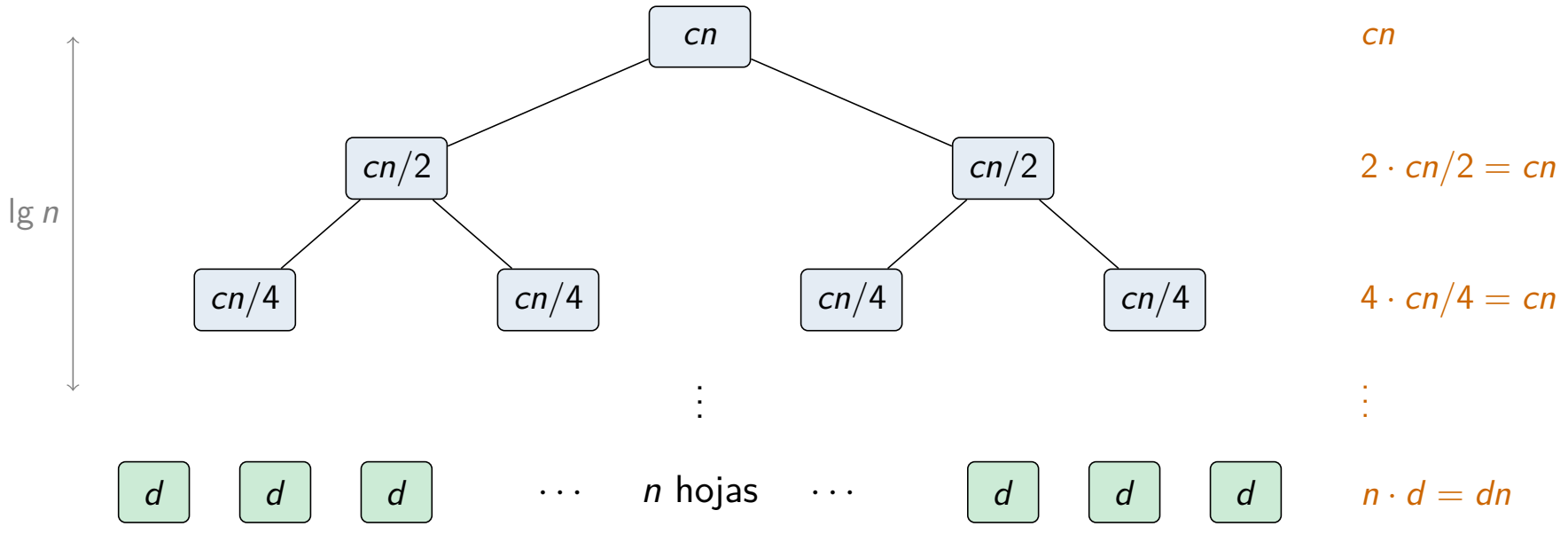
$2 \cdot cn/2 = cn$

$4 \cdot cn/4 = cn$

Pregunta

En el nivel i hay 2^i nodos de $cn/2^i$. ¿Cuántos niveles hay antes de las hojas, y cuánto suman las hojas?

$T(n) = 2T(n/2) + cn$: el árbol completo



Suma

lg n niveles internos de cn cada uno, más n hojas de d: $T(n) = cn \lg n + dn$, lo mismo que dio la sustitución.

$T(n) = cn \lg n + dn$: comprobación por inducción

Teorema

Si $T(1) = d$ y $T(n) = 2T(n/2) + cn$ para $n \geq 2$ potencia de 2, entonces $T(2^j) = c\,2^j j + d\,2^j$ para todo $j \geq 0$.

Estrategia

Se procede por inducción sobre el exponente j.

Desarrollo

Base ($j = 0$): $T(1) = d = c \cdot 1 \cdot 0 + d \cdot 1$.
Hipótesis: para un $j \geq 0$ fijo, $T(2^j) = c\,2^j j + d\,2^j$.

$T(n) = cn \lg n + dn$: comprobación por inducción

Desarrollo, continuación

Paso:

$$\begin{aligned} T(2^{j+1}) &= 2\,T(2^j) + c\,2^{j+1} \overset{\text{HI}}{=} 2\big(c\,2^j j + d\,2^j\big) + c\,2^{j+1} \\ &= c\,2^{j+1} j + c\,2^{j+1} + d\,2^{j+1} = c\,2^{j+1}(j+1) + d\,2^{j+1}. \end{aligned}$$

Conclusión

Por inducción, con $n = 2^j$, $T(n) = cn \lg n + dn$ para toda potencia de 2.

Errores comunes con recurrencias

Contar nodos en vez de trabajo

El nivel i del árbol de la mezcla tiene 2^i nodos, pero cada uno trabaja $cn/2^i$. El nivel suma cn, no $2^i cn$.

Olvidar las hojas

En $T(n) = 2T(n/2) + c$ los niveles internos suman $c(n-1)$ y las n hojas suman dn. Sin las hojas, la cuenta se queda corta.

n que no es potencia de 2

Con n cualquiera las mitades son $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$. El orden de crecimiento no cambia (CLRS, 3.ª ed., sección 4.6.2), y por eso se resuelve primero sobre potencias de 2.

Ejercicio 3: plantear, resolver y comprobar

Enunciado

1. (Base) Plantee la recurrencia de *maximo* con $m = fin - ini \geq 1$ y resuélvala por sustitución repetida.
2. (Intermedio) Plantee la recurrencia de *sumaMitades* con n potencia de 2, $n \geq 1$. Resuélvala con el árbol de recursión y compruebe la forma cerrada por inducción.
3. (Avanzado) En *UVa 839* cada línea de la entrada es un nodo del móvil. La función lee su línea, trabaja c y se llama sobre cada lado que sea un submóvil. Con n líneas y lados de a y b líneas, $a + b = n - 1$:

$$T(0) = 0, \qquad T(n) = T(a) + T(b) + c.$$

Demuestre que $T(n) = cn$ para todo $n \geq 0$, sea cual sea la forma del móvil.

Ejercicio 3: una pregunta sobre el punto 3

Pregunta

a y b pueden ser cualquier par con $a + b = n - 1$: 0 y $n - 1$, o 3 y $n - 4$. ¿Qué hipótesis cubre todos esos tamaños a la vez?

Ejercicio 3: una pregunta sobre el punto 3

Pregunta

a y b pueden ser cualquier par con $a + b = n - 1$: 0 y $n - 1$, o 3 y $n - 4$. ¿Qué hipótesis cubre todos esos tamaños a la vez?

Respuesta

La de la inducción fuerte: $T(j) = cj$ para todo j con $0 \leq j < n$. Como $a, b \leq n - 1$, cubre las dos llamadas sin importar cómo se repartan las líneas.

Solución del ejercicio 3, punto 1

Recurrencia

El caso base, $m = 1$, cuesta una constante d. El recursivo hace una llamada de tamaño $m - 1$ y una comparación con su asignación, de costo c:

$$T(1) = d, \qquad T(m) = T(m-1) + c \quad \text{para } m \geq 2.$$

Sustitución repetida

$T(m) = T(m-i) + ic$. El caso base es $m - i = 1$, o sea $i = m - 1$:

$$T(m) = T(1) + (m-1)c = c(m-1) + d.$$

Es la misma recurrencia de `suma` con la base en 1; el costo es lineal en m.

Solución del ejercicio 3, punto 2

Recurrencia

$T(1) = d$, $T(n) = 2T(n/2) + c$: dos llamadas de tamaño $n/2$ y, fuera de ellas, el cálculo de *mid* y una suma.

Árbol

El nivel i tiene 2^i nodos de costo c, para $i = 0, \ldots, \lg n - 1$, y hay n hojas de costo d:

$$T(n) = c\,(1 + 2 + \cdots + n/2) + dn = c\,(n-1) + dn.$$

Inducción sobre j, con $n = 2^j$

Base: $T(1) = d = c \cdot 0 + d$. Paso:
$T(2^{j+1}) = 2T(2^j) + c \overset{\text{HI}}{=} 2c(2^j - 1) + 2d\,2^j + c = c(2^{j+1} - 1) + d\,2^{j+1}$. Por inducción, $T(n) = c(n-1) + dn$: lineal, como `suma`, aunque parta el rango.

Solución del ejercicio 3, punto 3

Teorema

Si $T(0) = 0$ y $T(n) = T(a) + T(b) + c$ con $a, b \geq 0$ y $a + b = n - 1$, entonces $T(n) = cn$ para todo $n \geq 0$.

Estrategia

Se procede por inducción fuerte sobre n.

Desarrollo

Base ($n = 0$): $T(0) = 0 = c \cdot 0$.
Hipótesis: para un $n \geq 1$, $T(j) = cj$ para todo j con $0 \leq j < n$.

Solución del ejercicio 3, punto 3

Desarrollo, continuación

Paso: a y b son menores que n porque suman $n-1$, así que
$T(n) = T(a) + T(b) + c \overset{\text{HI}}{=} ca + cb + c = c\,(a + b + 1) = cn.$

Conclusión

Por inducción fuerte, $T(n) = cn$: la función visita cada nodo del móvil una vez, sea cual sea su forma.

Contenido

1 Inducción sobre las sumatorias del conteo

2 Correctitud de una función recursiva

3 Recurrencias: plantear y resolver

4 Referencias

Referencias

- K. H. Rosen. *Matemáticas Discretas y sus Aplicaciones*, 7.ª edición, McGraw-Hill. Capítulo 5: inducción matemática (5.1), inducción fuerte (5.2) y algoritmos recursivos con su correctitud (5.4). Sección 8.3: recurrencias de dividir y conquistar.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 3.ª edición, MIT Press, 2009. Apéndice A.1: sumatorias. Sección 2.3.2: la recurrencia de la mezcla y su árbol. Secciones 4.3 y 4.4: sustitución y árbol de recursión. Sección 4.6.2: pisos y techos.